GESTIÓN DE SISTEMAS DE DATOS MASIVOS

Information Management

01. Scalability in relational Database

02. Data Warehouse

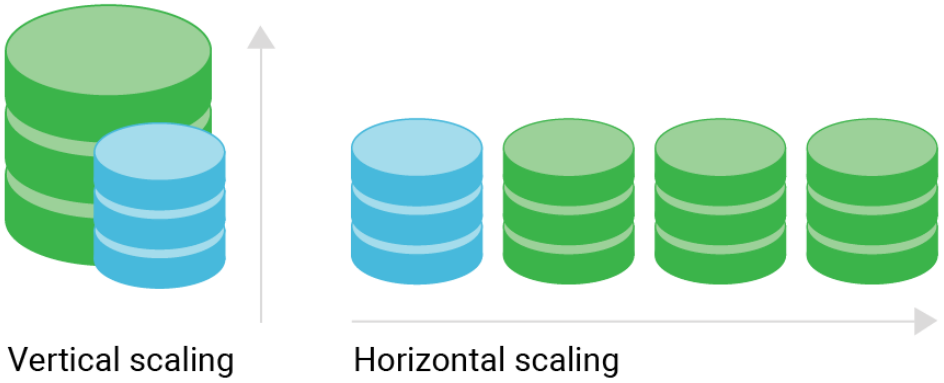
03. Star Schema

04. Snowflake Schema

05. Apache Hive Ecosistem

Scalability in relational Database

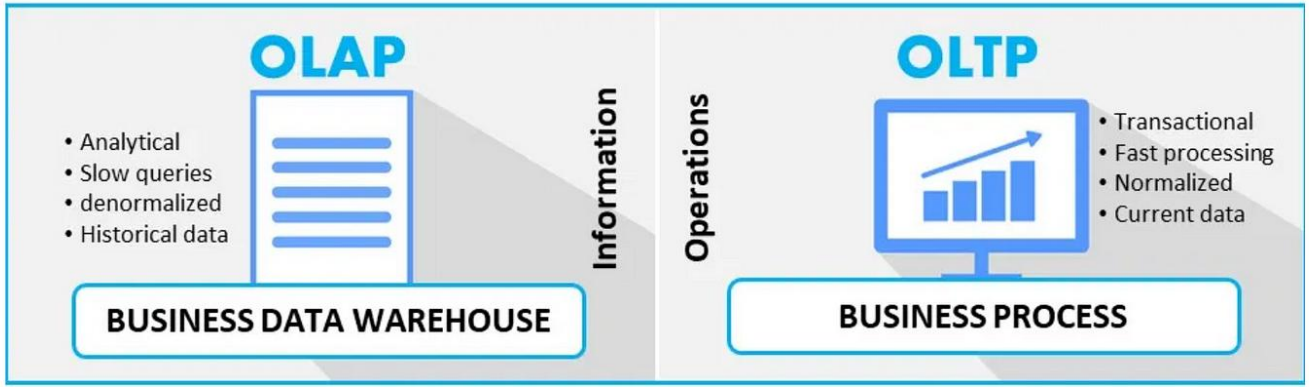
- **Scalability** in databases can be achieved in two main ways:
  - **Vertical Scalability**: This involves improving the hardware (upgrading servers, storage, etc.) and transferring the data to more powerful systems to handle increased loads.
  - **Horizontal Scalability**: This consists of adding more database instances and distributing the queries across them, simulating a unified system by managing connections between the different databases.



OLAP vs OLTP

- **OLTP (Online Transaction Processing)** focuses on handling day-to-day transactional tasks such as inserting, updating, and deleting data in real time. It is optimized for speed and efficiency in operations like retail transactions or banking.

- **OLAP (Online Analytical Processing)** is designed for complex data queries, aggregations, and analysis, often dealing with large datasets over time. OLAP is used for reporting, trend analysis, and business intelligence. While OLTP ensures real-time accuracy and data integrity, OLAP prioritizes query performance and deep analytical insights from historical data.



Data Warehouse VS Classic

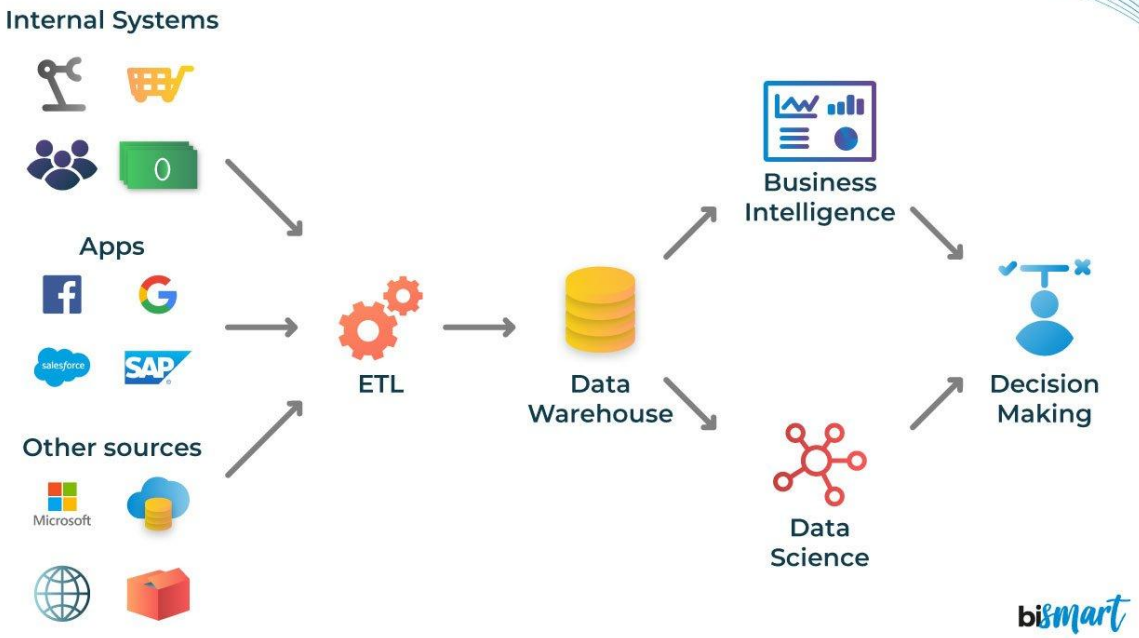
- A **classical database** is optimized for transactional processing (**OLTP**), handling frequent operations like inserts, updates, and deletions. It is highly normalized to maintain data integrity and is commonly used in real-time applications like banking or e-commerce.

- A **data warehouse**, on the other hand, focuses on analytical processing (**OLAP**) for querying and analyzing large volumes of historical data. It supports complex queries, aggregations, and trends, making it ideal for business intelligence. Unlike classical databases, data warehouses are denormalized to enhance query performance and scalability.

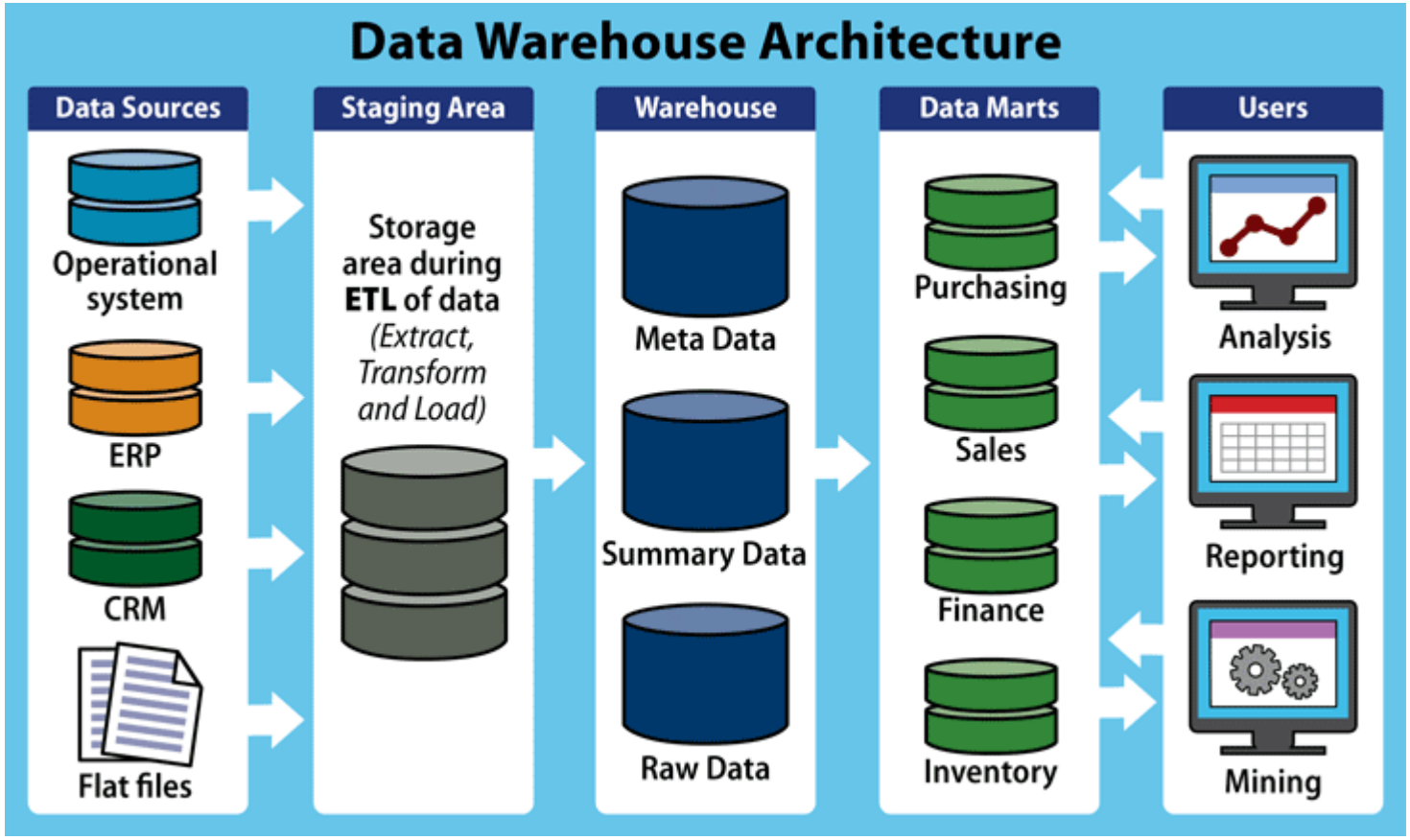
- While classical databases prioritize real-time transactions and consistency, data warehouses are designed for efficient analysis and high-speed querying of large datasets.

Data Warehouse

A **data warehouse** is a centralized repository that stores large volumes of structured data from multiple sources, designed to support business intelligence activities, reporting, and data analysis (**OLAP**).



Data Warehouse (Business)



Data Warehouse (Paid)

- **Amazon Redshift**: A cloud-based data warehouse service that allows for fast and scalable analysis.

- **Google BigQuery**: A fully managed data warehouse that enables analysis of large volumes of data.

- **Snowflake**: A cloud-based data warehouse platform that offers scalability and flexibility in data management.

- **Microsoft Azure Synapse Analytics**: A data analytics solution that combines data storage and processing.

- **Oracle Exadata**: A hardware and software platform from Oracle for running database applications and data warehouses.

Data Warehouse (Free)

- **Apache Hive**: A data warehousing solution built on Hadoop that allows SQL-like querying.
- **Apache Druid**: An analytical data warehouse designed for real-time event analysis.
- **PostgreSQL**: A relational database management system that can be used as a data warehouse with appropriate extensions.
- **Apache Kylin**: An OLAP platform for analyzing large datasets in Hadoop.
- **Greenplum Database**: A database based on PostgreSQL, optimized for large data set analysis.

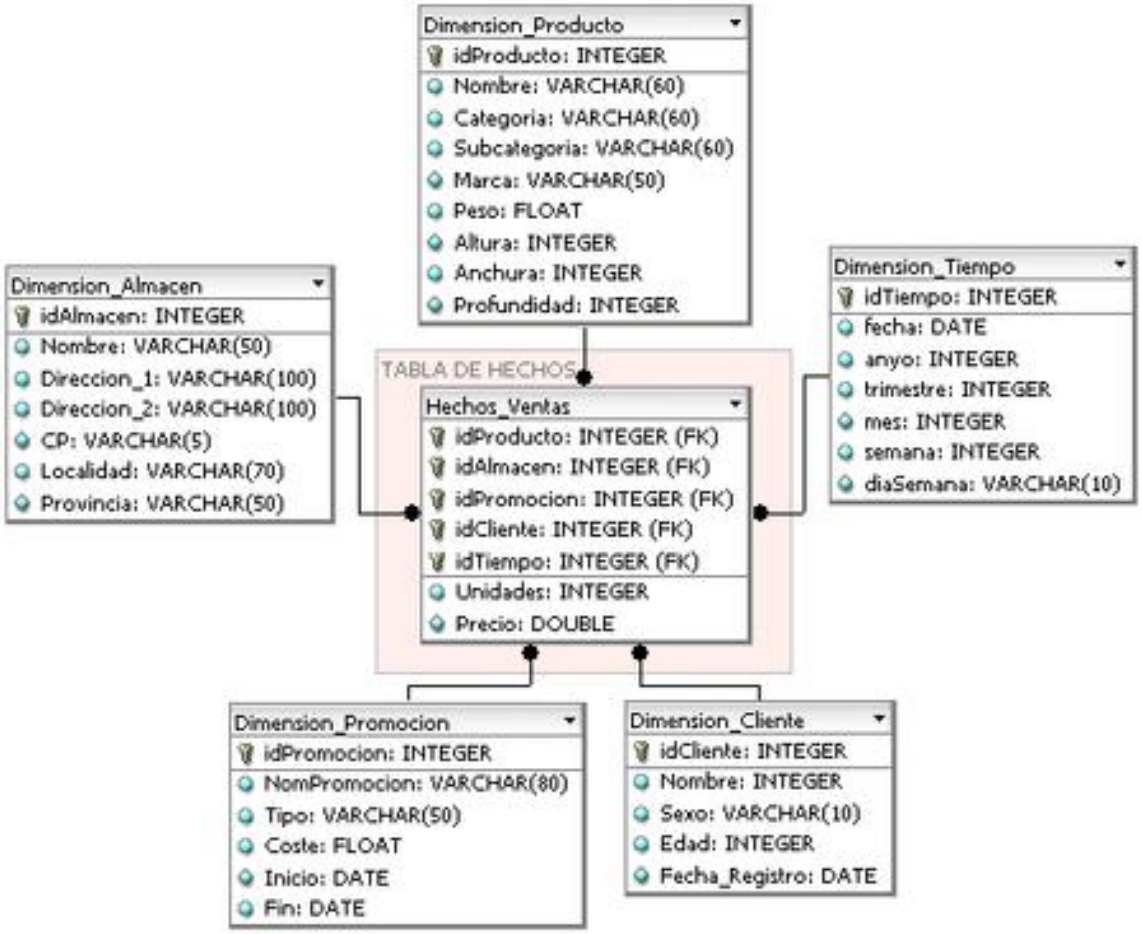
Star Schema

The **star schema** is a simple and widely used database schema in data warehousing. It consists of a central fact table that stores transactional data, surrounded by dimension tables that provide context to the facts (e.g., time, product, location). Key characteristics of the star schema include:

1. Single-level structure with a central **fact table**.
2. Dimension tables are directly linked to the **fact table**.
3. **Simple and efficient for querying**, especially for OLAP systems.



Star Schema

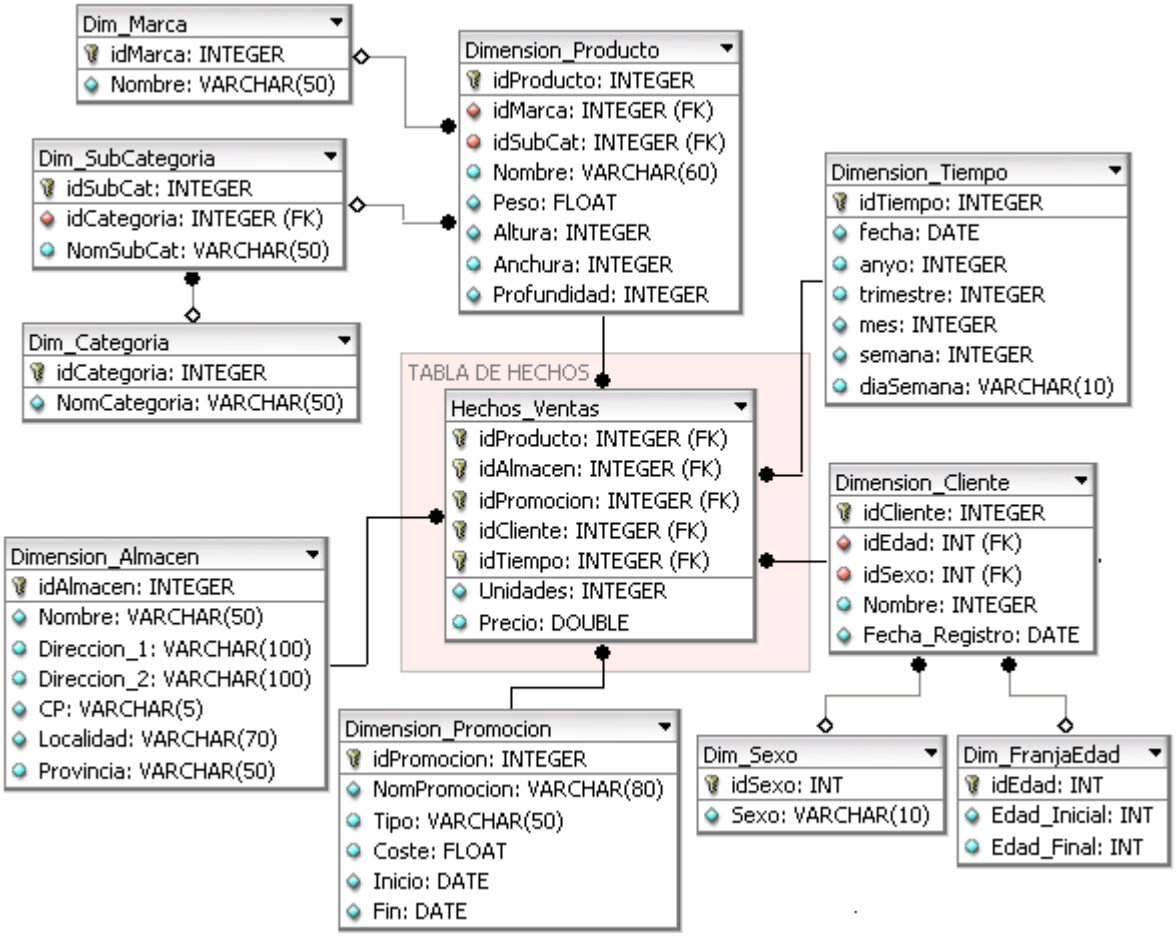

Dimension_Producto
idProducto: INTEGER
Nombre: VARCHAR(60)
Categoria: VARCHAR(60)
Subcategoria: VARCHAR(60)
Marca: VARCHAR(50)
Peso: FLOAT
Altura: INTEGER
Anchura: INTEGER
Profundidad: INTEGER
Dimension_Almacen
idAlmacen: INTEGER
Nombre: VARCHAR(50)
Direccion_1: VARCHAR(100)
Direccion_2: VARCHAR(100)
CP: VARCHAR(5)
Localidad: VARCHAR(70)
Provincia: VARCHAR(50)
Dimension_Tiempo
idTiempo: INTEGER
fecha: DATE
anyo: INTEGER
trimestre: INTEGER
mes: INTEGER
semana: INTEGER
diaSemana: VARCHAR(10)
TABLA DE HECHOS
Hechos_Ventas
idProducto: INTEGER (FK)
idAlmacen: INTEGER (FK)
idPromocion: INTEGER (FK)
idCliente: INTEGER (FK)
idTiempo: INTEGER (FK)
Unidades: INTEGER
Precio: DOUBLE
Dimension_Promocion
idPromocion: INTEGER
NomPromocion: VARCHAR(80)
Tipo: VARCHAR(50)
Coste: FLOAT
Inicio: DATE
Fin: DATE
Dimension_Cliente
idCliente: INTEGER
Nombre: INTEGER
Sexo: VARCHAR(10)
Edad: INTEGER
Fecha_Registro: DATE

Snowflake Schema

The **snowflake schema** is a more complex variation of the star schema. It involves normalization of dimension tables into multiple related tables, resulting in a "snowflake" shape. Key characteristics of the snowflake schema include:

1. **Multi-level structure** with **normalized dimension** tables.
2. **Reduced redundancy** in data storage.
3. **More complex queries** due to additional joins but improved data integrity.

Snowflake Schema



Apache Hive



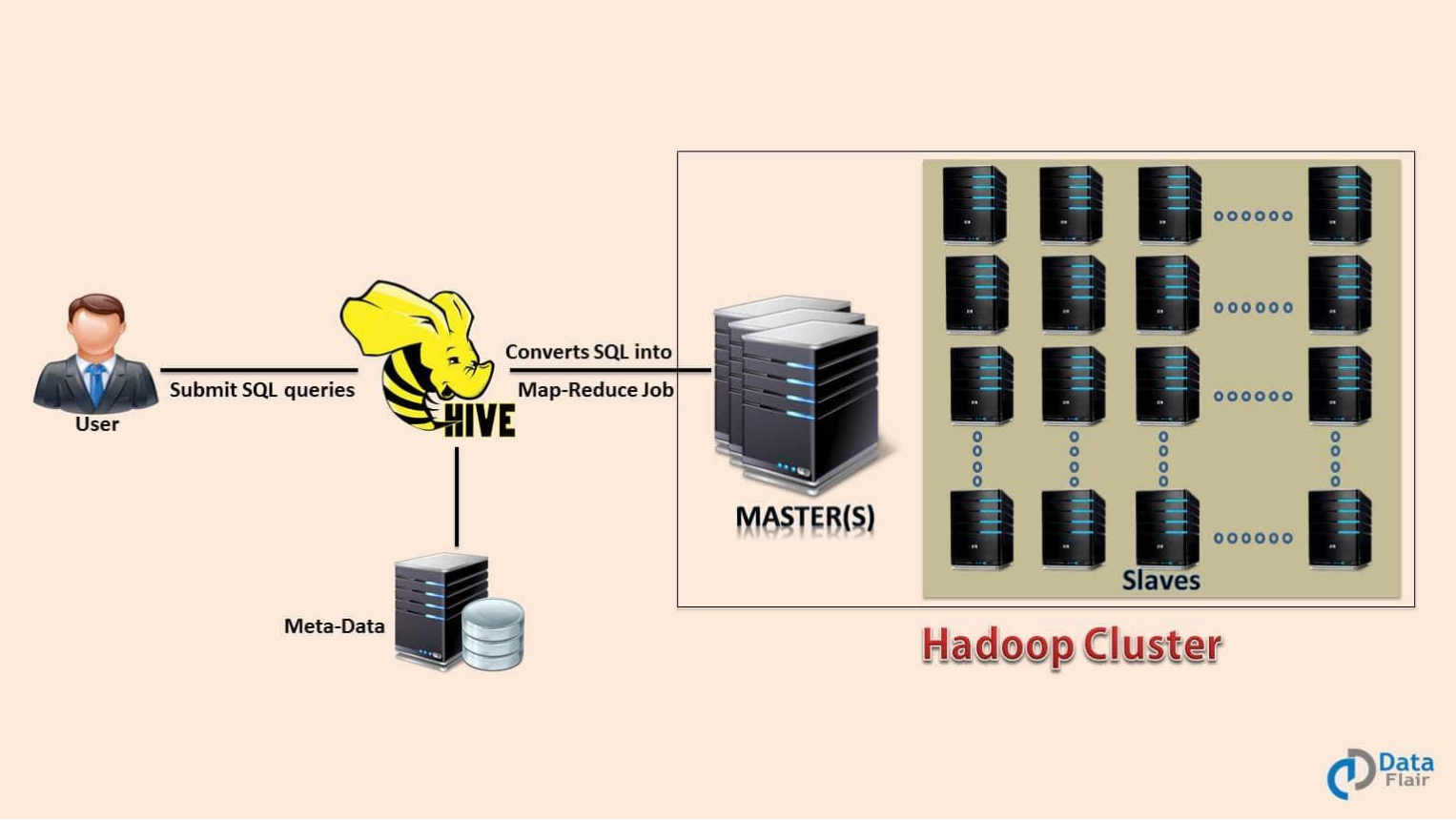
Apache Hive is an open-source data warehousing solution built on top of Hadoop, designed to manage and analyze large datasets stored in distributed systems.

It provides a SQL-like query language called HiveQL, which allows users to query and manage structured data efficiently.

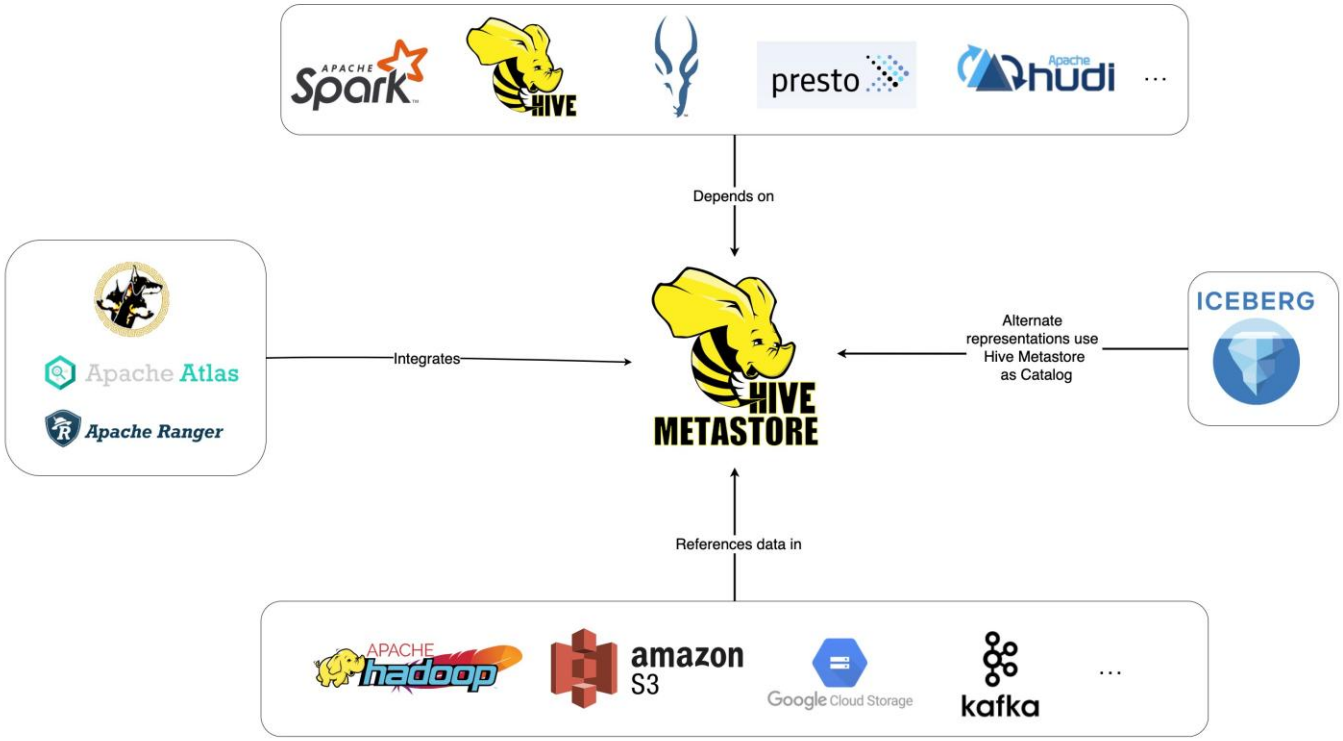
Hive acts as an infrastructure for data warehousing by enabling the integration of structured data from multiple sources, transforming it into a unified format for querying and reporting.

Its architecture supports batch processing and is optimized for reading, aggregating, and analyzing massive amounts of data, making it ideal for big data analytics and business intelligence tasks.

Apache Hive


User
Submit SQL queries
HIVE
Converts SQL into
Map-Reduce Job
MASTER(S)
Slaves
Meta-Data
Hadoop Cluster
Data Flair

Apache Hive


Depends on
Integrates
Alternate representations use Hive Metastore as Catalog
References data in
HIVE METASTORE
Apache Atlas
Apache Ranger
ICEBERG
amazon S3
Google Cloud Storage
kafka

Apache Hive



- Hive Server 2 (**HS2**): A service that executes queries by processing client requests and generating YARN jobs.
- **Hive ACID**: Provides full ACID support for ORC tables, enhancing data ingestion and updates.
- Hive Metastore Server (**HMS**): Acts as a central metadata repository for Hive tables and partitions, accessible via API.
- **Hive Replication**: Supports incremental replication for backup and recovery of data.
- **Hive Beeline Shell**: A command-line interface for executing **HiveQL** statements and queries from various applications.
- **Hive LLAP**: Enables subsecond interactive SQL queries, improving performance for data retrieval.
- **Security and Observability**: Integrates Kerberos authentication with Apache Atlas and Apache Ranger for enhanced data security.

Apache Hive(HiveQL vs SQL)

- **Nature**:
  - HiveQL: Designed to work with large volumes of data in a distributed storage environment like Hadoop.
  - SQL: A standard query language used in relational database management.

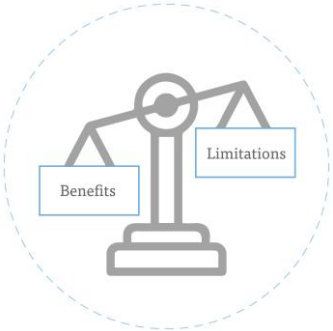
- **Data Model**:
  - HiveQL: **Optimized for structured, semi-structured, and unstructured data**; does not require a fixed schema.
  - SQL: Designed for strictly structured data with a **fixed schema**, requiring all tables and columns to be defined in advance.

- **Data Handling**:
  - HiveQL: Analytical queries and batch processing; can efficiently **handle large datasets**.
  - SQL: Real-time transactions and ad-hoc queries; provides **fast performance** for smaller operations.

Apache Hive (HiveQL features/limitation)

- **Features**:
  - Supports SQL-like queries, making it **easier for users familiar with SQL** to transition.
  - Ability to **query large datasets stored in Hadoop HDFS**.
  - Allows the use of **custom functions and maps** for data processing. (very complex)

- **Limitations**:
  - **Higher latency** in query execution due to its batch-oriented design.
  - **Limited** transaction capabilities compared to SQL.



Apache Hive (HiveQL language limitation)

- **Limited Support for Transactions**: HiveQL has restricted transactional capabilities, mainly supporting **batch processing** and lacking robust transaction management features like row-level locking.

- **Less Rich Query Language**: HiveQL does not support advanced SQL features such as window functions, recursive queries, and common table expressions (CTEs), limiting complex analyses.

- **Limited Data Types**: HiveQL offers a **narrower set of data types** compared to SQL, which can restrict handling specialized or custom data types.

- **Join Limitations**: While HiveQL supports joins, it may **perform poorly** with large datasets or complex joins, unlike SQL databases optimized for such operations.



Apache Hive (HiveQL language limitation)

- **Error Reporting**: HiveQL's error **messages are less informative** than those in SQL, making it harder to identify and resolve query issues.
- **Dependency on Hadoop**: HiveQL relies on the Hadoop ecosystem for execution, introducing **overhead and latency** not present in direct SQL executions.
- **Schema Evolution**: **Modifying schemas in HiveQL can be more cumbersome** than in SQL databases, where changes can often be made dynamically with minimal disruption.
- **Non-Real-Time Processing**: HiveQL is primarily **designed for batch processing**, while SQL is optimized for real-time data manipulation and quick query responses.

Apache Hive (GESDB Docker enviroment)

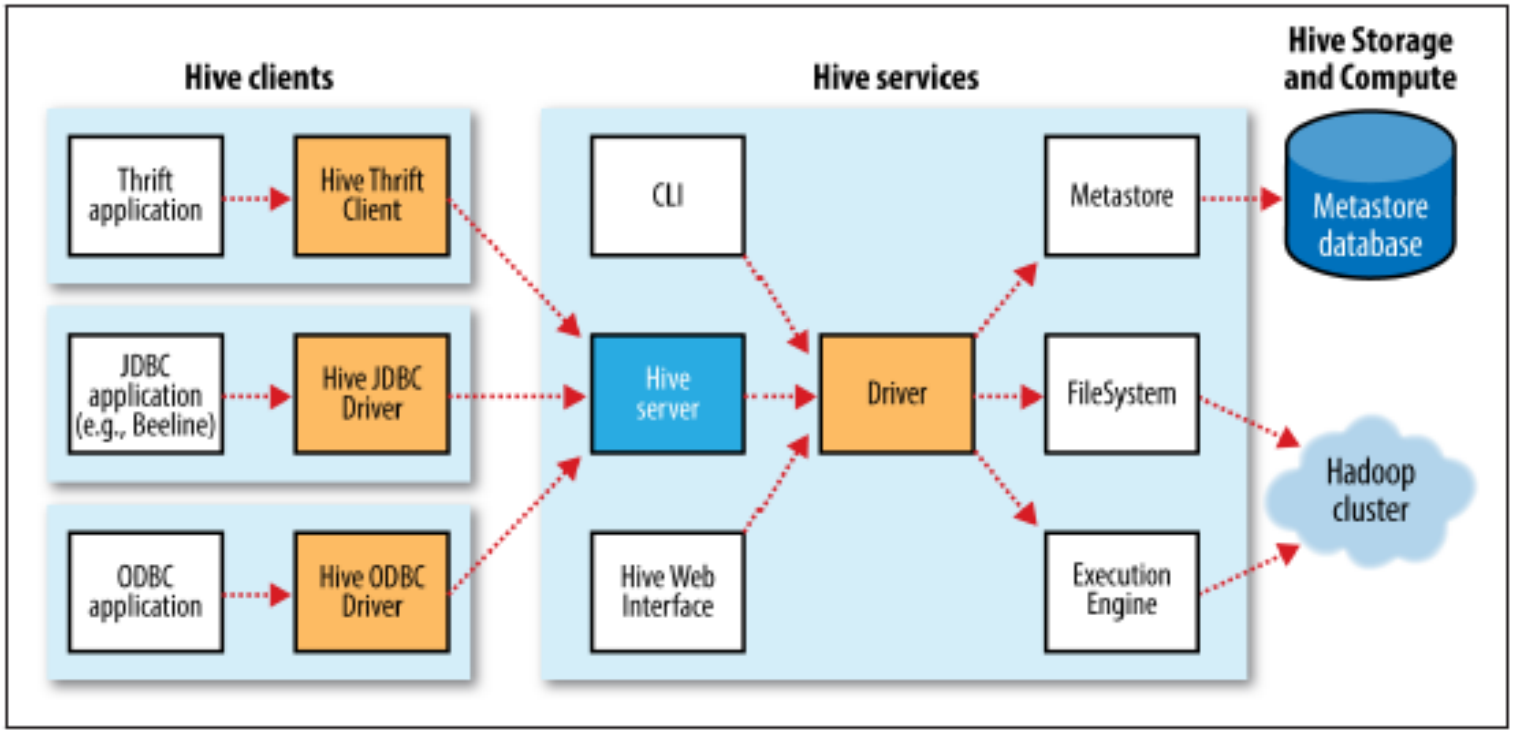


Figure 17-1. Hive architecture Source:*Hadoop: The definitive Guide*

Apache Hive (GESDB Docker environment)

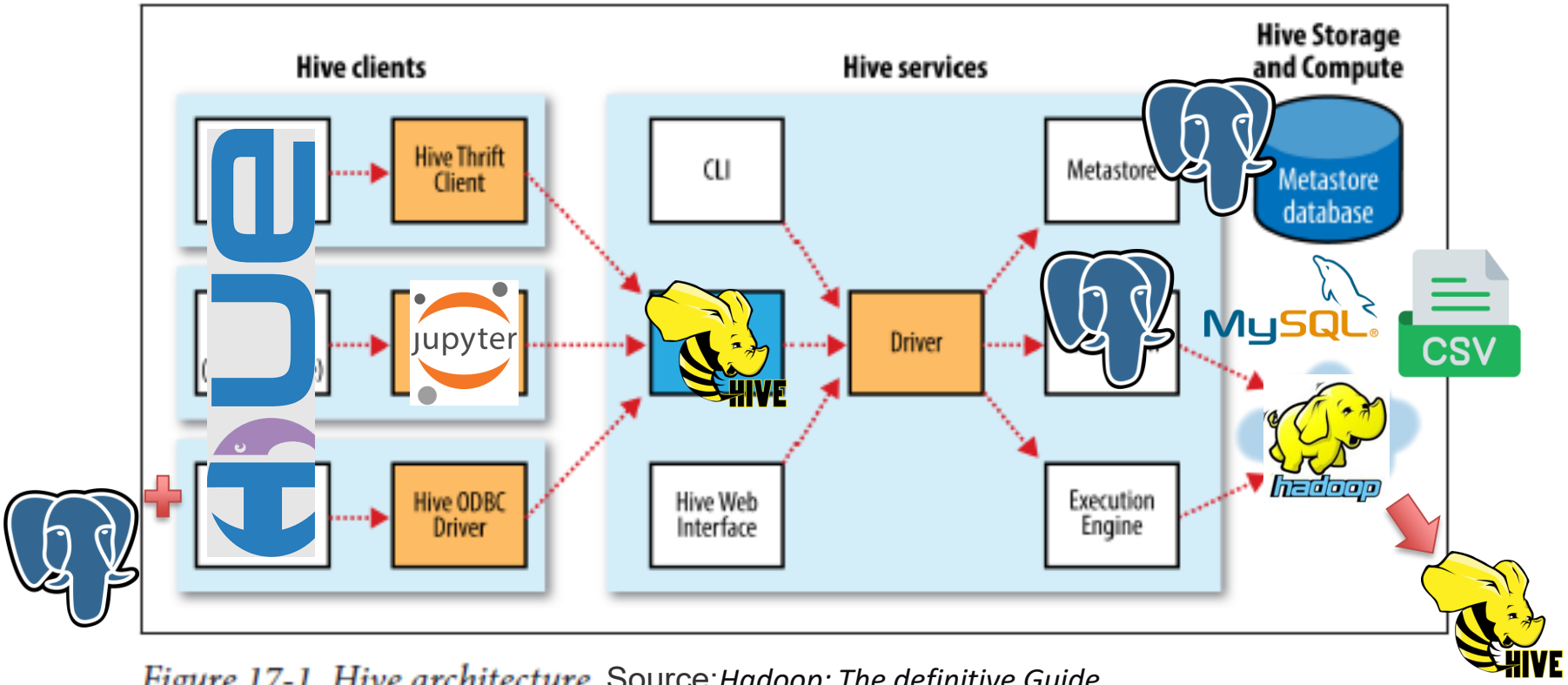
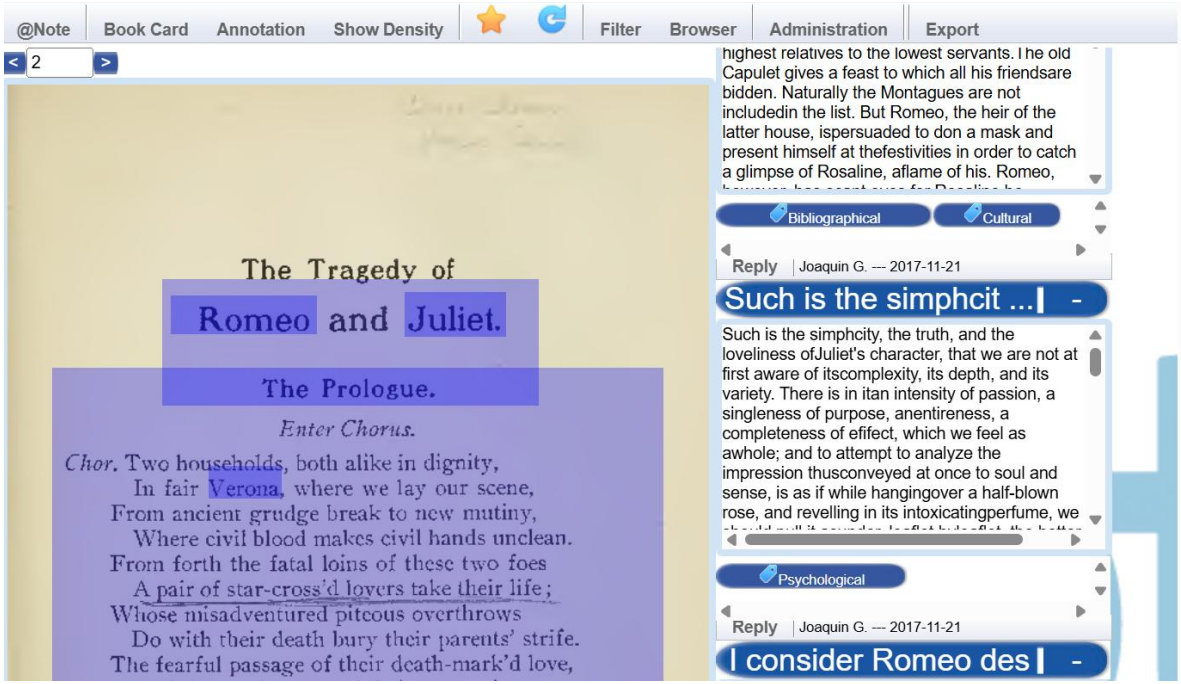


Figure 17-1. Hive architecture Source:*Hadoop: The definitive Guide*

Exemple Projects (@Note)

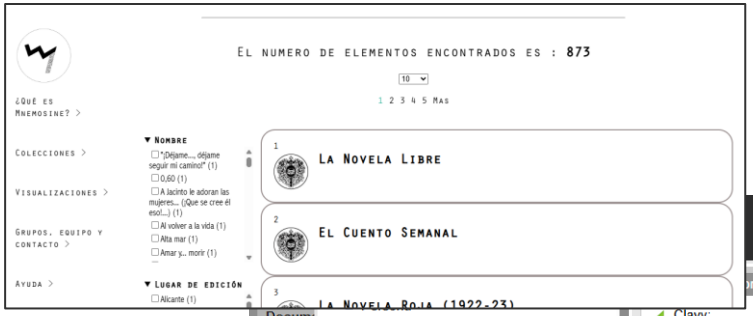


- **Unify three sources:**
  - PDF/Online Books
  - XML Structures
  - Free Relational Annotation
- **Lines of Work:**
  - Automatic Annotation
  - Mixed Source Annotation (Image/Text)
  - Mixed Notes (Audio, Video, Text)

- **http://a-note.fdi.ucm.es/**

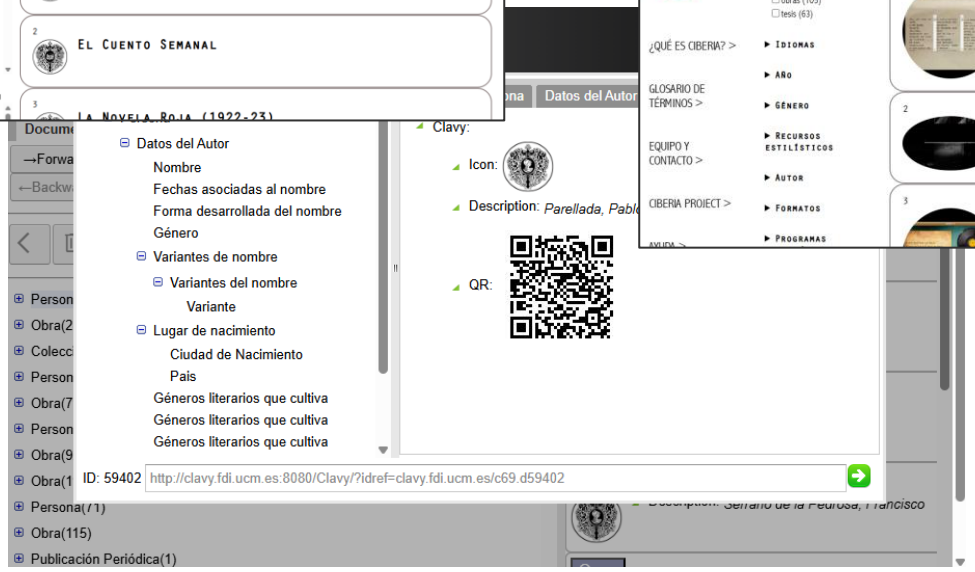
Example Projects (Clavy)

- **http://repositorios.fdi.ucm.es/mnemosine/**



- **http://repositorios.fdi.ucm.es/ciberia/**





- **http://clavy.fdi.ucm.es/**

Example Projects (Clavy)

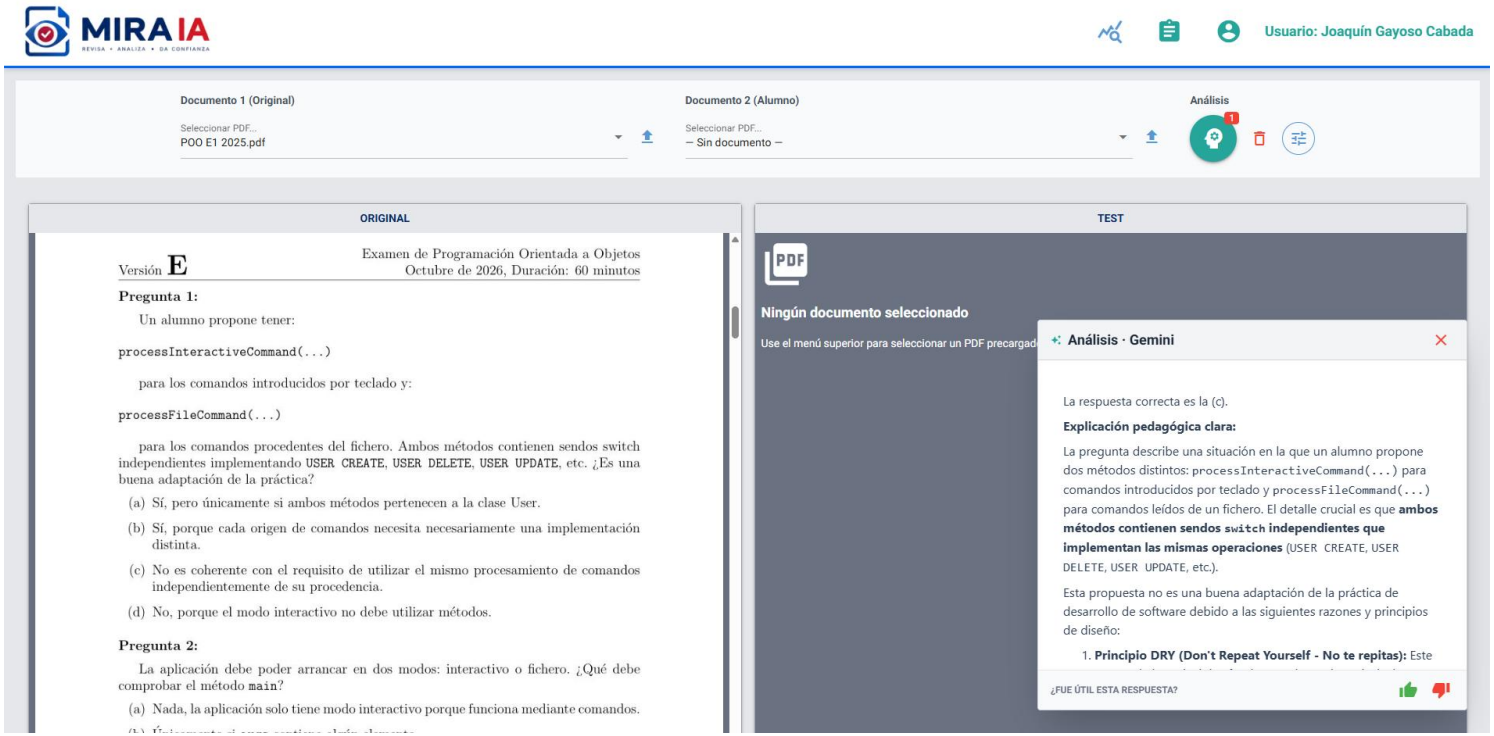
- **ETL System for the incorporation of an agnostic model.**
  - Separates the **documents** from the **structure that governs them**
    - **Attribute–Value Documents**
    - **Hierarchical Attribute Structure** in tree form
  - Uses the **structure** as a means of **transformation** and **expert editing**.
  - Uses **compiler techniques** for **processing the structure** and **exporting it as services**.
  - **Plugin system** to add services for **data ingestion, export, editing, and transformation**.
- **Lines of Work:**
  - **Storage optimization.**
  - **Decoupling** of **data downloading** and **use of remote live data.**
  - **Generation of a transformation language based on grammars.**

Example Projects (MiraIA)

MIRA IA
REVISA • ANALIZA • DA CONFIANZA

- **Unify two sources:**
  - Exam images
  - CSV solutions
  - CSV student data
- **Lines of Work:**
  - **Microservice-based Architecture**
  - **Automatic/Semi-automatic Exam Annotation**
  - **Fully Automated Evaluation Pipeline:** generation, evaluation, correction, and revision
  - **Integration of AI-assisted scoring and feedback generation**
  - **Standardization of exam data formats for interoperability**

Example Projects (MiraIA)



Example Projects (Sonar)



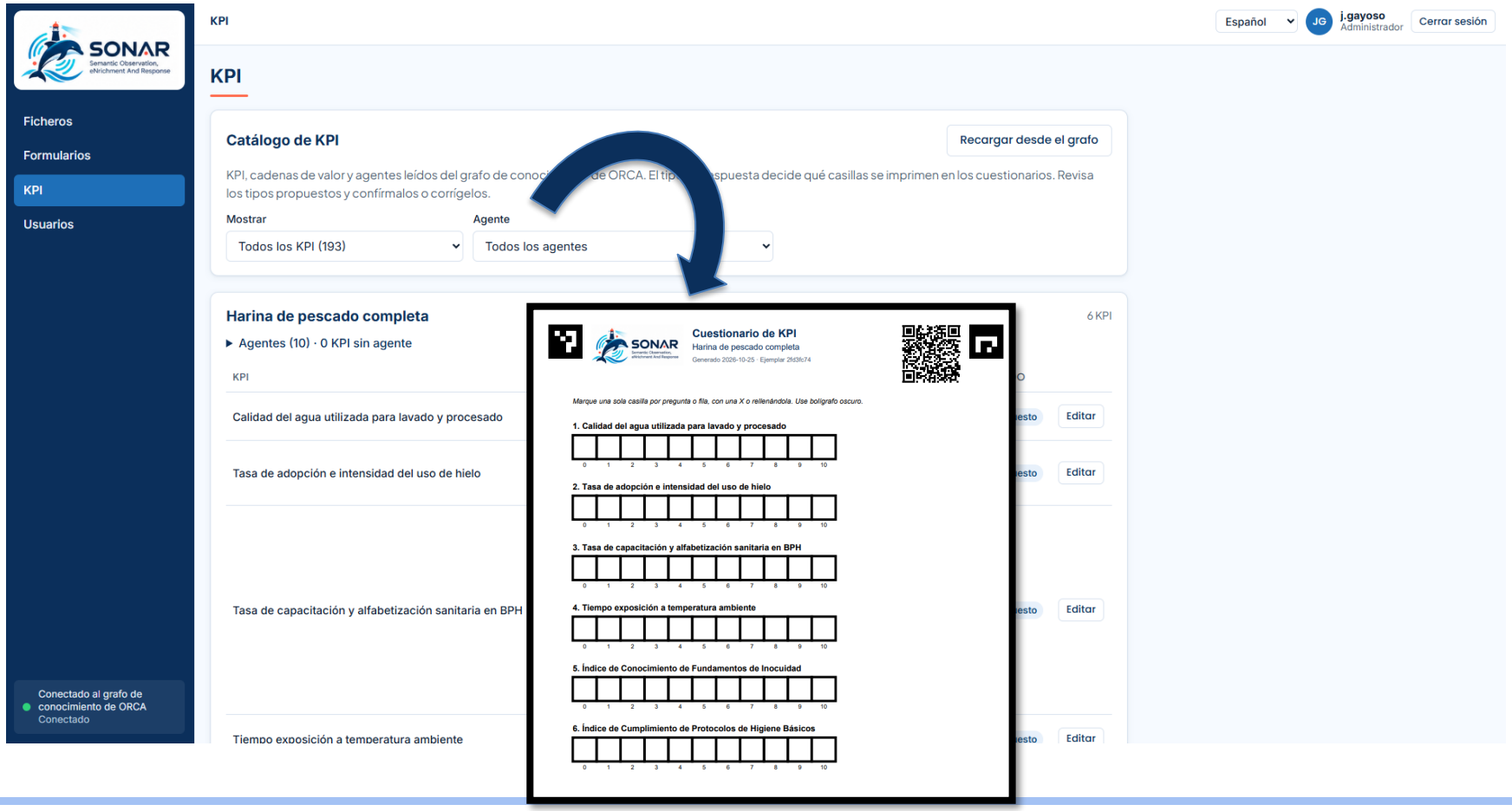
- **Connect two knowledge flows:**
  - ORCA Knowledge Graph
  - Survey data from fieldwork
- **Input Sources:**
  - Survey images
  - CSV files
  - XLSX spreadsheets

- **Lines of Work:**
  - Knowledge Graph-driven Survey Generation
  - Automatic/Semi-automatic Survey Data Extraction
  - Semantic Mapping and Entity Linking
  - Knowledge Graph Population from Survey Responses
  - AI-assisted Interpretation of Structured and Unstructured Responses
  - Data Validation, Traceability, and Provenance
  - Standardization of Survey Data Formats for Interoperability

- **End-to-End Pipeline:**

Knowledge Graph → Survey Generation → Data Collection → Data Extraction → Semantic Linking → KG Enrichment

Example Projects (Sonar)





POLITÉCNICA

UNIVERSIDAD
POLITÉCNICA
DE MADRID



Universidad
Politécnica
de Madrid

ETSI **SISTEMAS INFORMÁTICOS**